Generative AI Acknowledgement (why?)

All of the text in this thesis excluding Appendix B and *this* acknowledgement is be the output of Anthropic's Claude series of LLMs (Opus 4.6, Sonnet 4.5, and Haiku 4.5)[1] that I (Joey Perrello) obtained while working with Anthropic's coding agent Claude Code.[2] This acknowledgement exists because I (and my advisors, Adam and Ram) believe there should be full disclosure of generative AI when used in the creation of academic work, as some readers may be morally opposed to generative AI. Previous academic work went uncredited into the training of the models, so it is the responsibility of academics to attempt to show respect to those who contributed before by acknowledging use of LLMs. My advisors recommended this thesis' use of generative AI under the conditions that there was a *human-written* acknowledgement and an appendix entry detailing how the system works. The resulting system is an agent-assisted research flow that is systematic and comprehensive, and an integral aspect of this thesis' creation. I originally intended Saturn, the protocol discussed in this thesis, to be a Master's project, where the guidelines for completion were more relaxed than a Master's thesis. I decided at the beginning of the academic quarter to transition to a thesis. During this time I became inspired by the agent driven development work of Geoffrey Huntley (of Ralph loops), Dexter Horthy (of 12-factor agents), and Steve Yegge (of Gas Town). This inspiration spawned the idea of an academic research assistant with the same knowledge and comprehension of Saturn as my own. However, this system ended up requiring the same level of attention and focus as Saturn, with similar levels of complexity. The system and all use cases of LLMs in this thesis is described in Appendix B.

Appendix B (how)

This appendix will explain the use of LLMs in the completion of this thesis, and during the development of Saturn. This section, along with the Generative AI acknowledgement are entirely hand written me. Every other section of this thesis is the output of a LLM, specifically, Anthropic's Claude series of models. The code for Saturn and the thesis are visible on their respective Github repositories and is entirely the output of these large language models. The methods behind generating this code is described below. These methods are my best attempt to steer LLMs to be more reflective of my true intentions, design patterns, and beliefs. This system provided me with an agent that could serve as an agentic advisor, going beyond the guidance of one hour weekly sessions with the primary research advisor. This agentic

[1] https://www.anthropic.com/system-cards
[2] https://code.claude.com/docs/en/overview

workflow is effective at designing a thesis, retrieving facts and claims from memory, and correcting itself. This system hopes to provide a level of sophistication for papers that use generative AI in their creation.

Design Fictions

Design fictions are an existing practice of creating fictional scenarios where a non-existing idea becomes real, with the goal of creating speculative and provocative discussions about the idea. Design fictions have been used in debate for decades, however they provide a unique advantage with LLMs to create specification documents for a system. Design fictions provide a working example of an idea that can be reverse engineered by an LLM. The design fictions for Saturn can be found in the fiction directory of the Saturn Github repository. They follow the story of two fictional characters, Derek and Mira, and a fictional corporation: Megalink. Derek's role was a software engineer and self-proclaimed "home network wizard," meant to reflect a technically proficient integrator of Saturn who has configured Saturn on their network. The Megalink corporation is the company that created the system that would ultimately become Saturn. Mira symbolizes a non-technical user. She is the sister of Derick who visits and uses Saturn on Derick's network to generate AI captions on a fictional photo app. She didn't need to pay for anything or configure settings, she just joined Derek's network and she had new features that were not present before. Later, a new character, Jordan, was created to inspire the beacon system. Jordan was a developer who had leaked his company's API key from a Saturn server. Beacons were created to provision short lived keys on the network that are deleted after a short window. As said before, when a LLM reads this design fiction, it isn't interpreted as speculation, it instead becomes a vision that can be broken down into specs. The three characters provide the LLM with examples of who in the real world would use Saturn. There is an understanding that the end user (Mira) and the programer who configures Saturn on their network (Derek) require different types of documentation and tools. These design fictions lead to the initial specifications for the Saturn protocol and first implementations.

Documentation Website

During development of Saturn, a human and agent facing documentation website was created with the intent to be referenced to a coding agent in future context. Myself, or any other person could visit the website and use it as a reference for their coding agents, as it served as the main knowledgebase for Saturn. In fact, the documentation site has a llms.txt file specifically designed to guide AI agents. The documentation is split across three different sections: the user guide, the integrator guide, and an integrations page. The user page provides a high level overview of the system, and a short guide for discovering Saturn services on the network. I purposefully tried to avoid technical jargon such as mDNS,

TXT records, and endpoints. This page can therefore serve as an introduction to new users of the system, or it can be injected into the prompt for an agent for context about Saturn. The integrator guide provides the technical overview of Saturn and is meant to be read by developers and network administrators. This page includes how to configure a custom Saturn service, which endpoints were required, and Saturn SDK guides. This page provided basic context to agents implementing Saturn. The last page showcased integrations of Saturn in hopes of growing popularity for the protocol. If I designed a page that could prove how easy it is to implement Saturn as a protocol, it could become more popular and therefore standardized. Coding agents could be fed this site as context to understand how similar platforms have integrated Saturn in the past. Integrators or coding agents could submit their personal integrations of Saturn on a form on this page, which then become Github Issues for the website. This website was frequently used as a source of knowledge that was fed to an agent upon fresh work sessions. Without it, Saturn would need to be explained every session, risking the loss of implementation details and worse context for the agent. This webpage later served as an initial source of memory for the Moons system, which will be described later in the appendix.

Integrations into various applications

An important goal of Saturn was to create multiple interoperable technical prototypes built around the protocol _saturn._tcp.local, demonstrating that widespread adoption was feasible. Initially Saturn was integrated into the chat based AI applications OpenWebUI and Jan, later moving to the coding agent OpenCode. All these platforms were natively based in AI, meaning some readers or developers may disregard Saturn entirely if they are working on projects or apps that are not AI native. Adam and Joey believed that Saturn made AI possible in apps that were not AI native, and to prove this, Joey created a VLC extension that roasted a user based off of what media was playing. There was no chat, and no mention of AI whatsoever. Developing these integrations could have each been a master project on their own. However, Joey took advantage of coding agents to develop them faster instead. I was able to integrate to these platforms faster because they would often have thorough documentation or source code on the web to reference to a coding agent. This had a snowball effect where the more integrations got built, the more examples of Saturn could be given as context to future agent sessions.

Serve role as a “robotic” advisor

Joey and Adam met once a week on an hourly basis to discuss Saturn. Outside of that hour, the AskUserQuestion tool incorporated into Claude Code would assist in taking notes for writing the thesis

paper. I actively engaged with Claude Code to arrive at different claims for the thesis, organize background work, and create an evaluation plan by chatting, and answering questions. Initially, the agent wrote all of its memory of Saturn and the interviews in a single markdown file. This file was disorganized, lacked technical depth, or misinterpreted information I would reference. The agent struggled to remember previous conversations the student would have, and it would have difficulty piecing together the repository, previous academic work, and the claims of the system. There needed to be a system that gave agents the same level of knowledge I had about Saturn, while providing the ability to hyperfocus on specific sections of the thesis depending on the focus of a work session.

Moons

Moons is the memory system that served as the knowledge base for all coding agents while I wrote the thesis. Moons did not exist while Saturn was being developed, only during the writing process of my thesis. The name “moons” comes from Saturn, the planet, notoriously having a lot of moons. Each individual moon can be thought of containing one small piece of information about the greater and more complex Saturn. Moons is a JSON file that points to different directories and markdown files in the moons directory. Each entry in this JSON file has the following fields:
id: the element of saturn that the current chunk is describing. Example: claim1, beacons, ai-sdk
type: this describes what role the id has in the saturn project. Example: claim1 has a type of claim, and beacons has a type of concept.
Desc: A short one sentence description of what the element is. Example: ephemeral-keys description reads "Time-limited API credentials (10-min lifetime, 5-min rotation with overlap for zero-downtime transitions) distributed via mDNS TXT records and deleted on shutdown, paralleling Kerberos session-layer distribution. Implemented in Python (KeyManager with thread-safe locks) and Rust (full lifecycle with `Drop` impl), making the 'leaked secret never removed' class (Meli 2019: 81%) structurally impossible."
File: reference to where a detailed markdown file of the concept is. These files contain a more detailed explanation of the ID, where to find the source files related to the ID, and why it matters in the project.
Edges: list of other ids and their relation to the current chunk
Example moon chunk:

```JSON
{"id": "mdns", "type": "concept", "desc": "Link-local name resolution (RFC 6762) via multicast to 224.0.0.251:5353 requiring no DNS server — Saturn's protocol foundation, used identically to how Bonjour discovers printers, AirPlay,
```

```
and Chromecast. Key limitations: AP isolation blocks multicast (confirmed on eduroam/UCSC-guest Jan 2026),
passive eavesdropping exposes all announcements, and 59% of device names leak real user names.", "file":
"concepts/mdns.md", "edges": [
    {"to": "claim-1", "rel": "supports"},
    {"to": "dns-sd", "rel": "related"},
    {"to": "ch2", "rel": "discussed_in"},
    {"to": "ch3", "rel": "discussed_in"}
  ]},
```

The goal of moons is to create a knowledge graph for an agent. With moons, a conversation about Saturn is much easier because I could freely reference source material with my agent having a memory bank to reference. An example question would be: “What have we discussed in claim 2 so far, is there anything I am forgetting to add?” An agent can read the graph.json to find claim-2’s id and trace all the details for claim-2 through markdown files. I, the human in the loop, could see if my agent is forgetting any information, or hallucinating the results and claims I previously made. A drawback to this strategy is that the graph.json has to be read in its entirety every time moons are mentioned. This drawback helps provide Claude with a high level understanding of all of the components of Saturn and how they all connect, but ultimately fills the context window with irrelevant information. My alternative option was to query the graph via a script, but this strategy proved to be even more inefficient as one read call is cheaper than multiple bash calls considering Saturn’s graph.json was ~563 lines. The agent also has to have knowledge of the moons system each session, and the only way to effectively provide the knowledge of moons existence is via an AGENTS.md file or a CLAUDE.md file. This ended up taking 20 lines in my CLAUDE.md file which is already a context burden for most systems, but proved to not have a noticeable effect for this thesis.

```
None
### Moons (Knowledge Base)
This project uses a structured knowledge graph called `moons/` to organize all
thesis context.

**Structure:**
- `moons/graph.json` — the map. Lightweight nodes with typed edges. Read this
FIRST to navigate.
```

```
- `moons/concepts/` — Saturn concepts (mDNS, DHCP, ephemeral keys, etc.)
- `moons/claims/` — the 3 thesis claims with evidence
- `moons/papers/` — distilled paper summaries with relevance to Saturn
- `moons/voice/` — user's personal perspective, design rationale, interview
notes
- `moons/code/` — code component analyses with line citations
- `moons/chapters/` — chapter planning and section status
- `moons/meta/` — advisor notes, timeline, project context

**How to use:**
1. Always `Read(moons/graph.json)` first. At ~500 lines it's cheap to read in
full and gives complete graph visibility — all nodes, edges, and types in one
shot. This holistic view is intentional: it surfaces connections you weren't
looking for.
2. Follow edges to specific content files only when you need depth.
3. After interviews or new information, update both the relevant content file
AND graph.json edges.
4. Keep graph.json lean — short descriptions, not full content. Depth lives in
the files.
5. `moons/query.py` exists as a human CLI convenience (`python moons/query.py
edges-to claim-3`).

**Moons is the workspace. thesis.tex is the submission.** Moons organizes
thinking; thesis.tex is the final output.
```

I often discovered gaps of knowledge or evidence in Saturn by viewing the moons knowledgebase. I would notice certain chunks had less edges than I believed they should, leading me to discover I forgot to upload an implementation piece of Saturn to the thesis. Moons highlighted areas of Saturn that had the most evidence, and which areas needed more support or focus. Coding agents also had knowledge of how

moons worked, meaning they could create interactive websites for visualizing Saturn's ideas.

Finding Academic Papers

Google announced Scholar Labs during the research phase of Saturn. Google Labs is a search engine that is powered by Gemini, and is developed like most chat bots; a user can query in natural language and the model will find academic papers related to that question. With the knowledge stored in moons, I could ask

my agent to generate research questions for my thesis to be later sent to Scholar Labs. Once I had a result I would read the abstract, results, and figures of each paper before downloading and storing them inside the moon's knowledge base. The agent would always require approval for its integration of the paper, and this process was continuously iterated for finding more papers. The moon knowledge base grew as my own understanding and research behind Saturn grew.

AI contribution in the evaluation section

The second claim in the evaluation section of this thesis was that Saturn ultimately provided less cognitive overhead than traditional API configuration methods. To prove this, my strategy was to perform a cognitive walkthrough, where I documented a series of steps in order to convey the workload required to accomplish each configuration setup. The initial goal was for an agent to autonomously count all the steps with the MCP server Playwright in isolated Docker containers. This ended up failing as the system was unreliable and difficult for me to verify. Perhaps with the invention of OpenClaw I overlooked an entirely agentic system. Instead, I opted for a different approach that relied on my previous design fiction's three different audiences for Saturn. There are people who configure Saturn on the network, app developers incorporating Saturn, and end users who consume the ai services. I first performed a 100% human-written cognitive walkthrough of creating an API key on Openrouter and placing it in an environment variable file. These were shared steps that both a system with and without Saturn needed to take. After this initial cognitive walkthrough, I wrote python scripts from the design fiction's multiple perspectives:

- An app developer using saturn's python package to send a query to an api endpoint
- An app developer using the traditional method of querying an api endpoint (including Stripe billing)
- The cognitive walkthrough for a sys admin distributing api keys to their team
- The cognitive walkthrough of a sys admin configuring a saturn beacon on their network without using the saturn python package
- The cognitive walkthrough of a sys admin configuring a saturn beacon on their network using the saturn python package
- An end user using an app with saturn on their network
- An end user using an app that uses AI without saturn

Note: These python scripts are as minimal as possible and try to avoid excessive features. The end user scripts contain no runnable code, rather just a docstring of the walkthrough this fictional person went through.

Academic Writing Skill

LLMs played a pivotal role in generating the text written in the thesis document. I created Moons to solve the problem of giving the agent memory of Saturn, and I utilized the AskUserQuestion tool to conduct self-guided interviews on my thoughts and claims for the thesis. Still, I faced a consistent problem: AI agents continuously failed to write what I considered to be captivating and well-written academic papers. The papers themselves are long and flood context, not including the knowledge required to remember what arguments and evidence are required when writing the paper. Even with the moon's knowledge base, it was impossible to one-shot a well written thesis in a singular prompt. Dividing the thesis into context-manageable sections wasn't effective either as the agent would often repeat information referenced in other sections of the thesis. I also believe AI writing to be subpar and lacking substance, with most of its output content following the same structure that eventually becomes a glaring pattern. Additionally, models had the tendency to scatter knowledge with no cohesion. To solve this problem I created the academic-writing skill. An agent will invoke this skill when the user directly mentions it or the description matches the problem the user is asking the agent to solve. For the academic-writing skill my description read:

```
None
"Use this skill whenever writing prose, essays, academic content, reports, or any long-form written output, even if the user doesn't explicitly mention writing quality. Trigger for any drafting, revising, or editing of written sections — including when the user pastes text and asks for feedback."
```

The skill's body includes general writing skills that I personally believed to be best practice and reflected how I wanted the thesis to be written. The rules of writing were divided into the following sections: Concreteness, Reader-Centered Writing, Dialogic Framing, Structure, and Rigor. The body also included a core workflow of drafting, evaluating, revising, and presenting the revision. These rules and the workflow were written to be generalizable enough that they could be applied to any area of my thesis. I wanted individual sections of my thesis to have specific rules and guidelines for writing, so markdown files were created for rules on that specific section of the thesis. The main skill body directed the agent to read the markdown file associated with the section it was writing. Now, agents could invoke this skill to write or make changes to specific sections (like the abstract or evaluation) of the thesis. It had the rules to generally follow, but also the rules for writing specific areas. There is no mention of Saturn itself in this skill, as I wanted the agents to understand these rules applied to all types of academic writing and not Saturn in isolation.

AI Detection Ralph Loop

Ralph loops were invented by Geoffrey Huntley and they allow a user to run fresh agent sessions with the same prompt infinitely. A simple Ralph loop looks like this:

```
None

while :; do cat PROMPT.md | claude-code ; done
```

This trick allows agents to correct themselves and complete a long repetitive task autonomously. While I was writing my thesis, I noticed that no matter how many interviews or evidence there was in moons, the agent would still write vapid claims or fail to see the thesis as a whole piece of work, not individual sections. Initially, I planned to run Sapling, an ai detection software, inside a Ralph loop so an agent can receive a score and try to minimize the score for the next pass. Adam was strongly opposed to this idea, as there was no clear goal or structure behind beating the score. An agent could essentially game the system and not actually improve the quality of writing. So, Adam and Joey read through sections of the thesis and transcribed to an agent what they believed to be common mistakes and blunders the agent was making when writing. Claude then organized this transcription into “academic-advisor-notes.md.” These notes served as guidelines that a subagent was given to grade my thesis. After this session, I designed a new Ralph loop. The Ralph loop begins with an agent reading notes from the previous Ralph session, the academic advisor notes, and the moons graph. Then, this agent spawns a subagent that reads the content of the thesis and grades based on the guidelines of Adam and I. This read-only subagent returns the grade back to the main agent with specific sections annotated as needing more correction than others. The main agent uses the knowledge it previously read about, and the academic-writing skill, to rewrite the thesis. After the thesis is finished, the agent takes notes of the score the thesis received before edits and the changes it took to write those sections better. These notes have the goal of steering future agents towards some writing strategies and away from others. If the previous Ralph iteration made major changes but the score decreases in the next session, that future agent knows to avoid the tips that the old agent left notes for. This Ralph loop process ultimately led to the creation of the first thesis draft that I eventually tweaked section by section.

Structured Arguments Skill

The structured arguments skill was a skill used for an agent to organize my claims, evidence, reasoning, arguments, etc in a format that was modeled after Belcak et al. (2025) "Small Language Models are the Future of Agentic AI." This paper labels every claim, argument, and counter-argument with short IDs and cross-references them throughout. This was a formatting skill but ultimately made the thesis easier to follow while eliminating my burden of manually labelling.